

Norms in the Non-Uniform-Grid LADMM Scheme

Working note – Steps 1–6 of N (Step 3 corrected)

This is a working note, built incrementally. This draft covers only Step 1: pinning down, precisely, what norm is used where in the code today. Nothing here yet addresses what the norms should be – that is Step 2, once this step is confirmed.

Step 1: The two grids, and the norm each one currently uses

1a. Grid points, cell centers, and what lives where

Time edges $t_0 = 0 < t_1 < \dots < t_{n_t} = 1$; cell $j + \frac{1}{2}$ (for $j = 0, \dots, n_t - 1$) is the interval (t_j, t_{j+1}) , with width $\Delta t_{j+\frac{1}{2}} := t_{j+1} - t_j$. Space edges $x_0 = 0 < x_1 < \dots < x_{n_x} = 1$, uniform, $\Delta x = 1/n_x$; cell $i + \frac{1}{2}$ (for $i = 0, \dots, n_x - 1$) has width Δx . *Integer indices are always grid points (edges); half-integer indices are always cell centers* – exactly the convention the main paper already uses for the space marginals $\mu_{i+\frac{1}{2}}, \nu_{i+\frac{1}{2}}$.

- **Collocated variable** $y = (\rho, m)$ (touched by the KE prox): lives at every $(j + \frac{1}{2}, i + \frac{1}{2})$, $j = 0, \dots, n_t - 1$, $i = 0, \dots, n_x - 1$ – $n_t n_x$ points, cell-centered in *both* directions.
- **Staggered variable** $x = (\tilde{\rho}, b)$ (touched by the projection):
 - $\tilde{\rho}$ is staggered *in time*: it lives at $(j, i + \frac{1}{2})$ for the *interior* time edges $j = 1, \dots, n_t - 1$ (space still cell-centered), $i = 0, \dots, n_x - 1$ – $(n_t - 1)n_x$ points. The two boundary edges $j = 0, n_t$ carry the *fixed* data $\mu_{i+\frac{1}{2}} := \tilde{\rho}_{0,i+\frac{1}{2}}, \nu_{i+\frac{1}{2}} := \tilde{\rho}_{n_t,i+\frac{1}{2}}$ – not part of the optimization variable.
 - b is staggered *in space*: it lives at $(j + \frac{1}{2}, i)$ for time centers $j = 0, \dots, n_t - 1$, *interior* space edges $i = 1, \dots, n_x - 1$ – $n_t(n_x - 1)$ points. The two boundary edges $i = 0, n_x$ carry the fixed homogeneous Neumann data $b = 0$.

A consensus map $\mathcal{A}$ (time/space averaging) turns x into something living on the same $(j + \frac{1}{2}, i + \frac{1}{2})$ grid as y , so the ADMM constraint $\mathcal{A}x = y$ compares like with like. Nothing in this step needs $\mathcal{A}$'s formula – just that x and y genuinely live on different point sets.

1b. Where a norm is actually used

Three distinct places in the code involve a norm on one of these variables.

1. The x -update (projection, `prox_f1` in `ladmm.py`). Solves, for the staggered $(\tilde{\rho}, b)$, $x^{k+1} = \arg \min_{x \in \text{constraint set}} \|x - v\|^2$ with the **plain, unweighted Euclidean norm**:

$$\|x - v\|^2 = \sum_{j=1}^{n_t-1} \sum_{i=0}^{n_x-1} (\tilde{\rho}_{j,i+\frac{1}{2}} - v_{j,i+\frac{1}{2}}^{\tilde{\rho}})^2 + \sum_{j=0}^{n_t-1} \sum_{i=1}^{n_x-1} (b_{j+\frac{1}{2},i} - v_{j+\frac{1}{2},i}^b)^2,$$

no Δt , no Δx , anywhere.

2. The y -update (KE prox, `solve_y` in `pipeline_nu.py`, calling `prox_ke_cc_nu`). Solves, independently for *every* cell $(j + \frac{1}{2}, i + \frac{1}{2})$, $j = 0, \dots, n_t - 1$, $i = 0, \dots, n_x - 1$:

$$(\rho, m)_{j+\frac{1}{2}, i+\frac{1}{2}}^{k+1} = \arg \min_{\rho, m} \frac{\Delta t_{j+\frac{1}{2}}}{\Delta t_{\text{ref}}} J(\rho, m) + \frac{\gamma}{2} [(\rho - \rho_{j+\frac{1}{2}, i+\frac{1}{2}}^z)^2 + (m - m_{j+\frac{1}{2}, i+\frac{1}{2}}^z)^2],$$

$$\Delta t_{\text{ref}} := \frac{1}{n_t} = \frac{1}{n_t} \sum_{j=0}^{n_t-1} \Delta t_{j+\frac{1}{2}}.$$

The *penalty* half (the $\frac{\gamma}{2}[\dots]$ term – the actual distance being minimized) is again the **plain, unweighted Euclidean norm** on the single cell $(\rho, m) \in \mathbb{R}^2$. Only the *objective* J picks up a weight, and it is the *relative* $\Delta t_{j+\frac{1}{2}}/\Delta t_{\text{ref}}$ (dimensionless, averages to 1 over $j = 0, \dots, n_t - 1$), not the absolute physical $\Delta t_{j+\frac{1}{2}} \Delta x - \Delta x$ never appears. (This looks like a weaker, ad hoc substitute for the absolute weight – it isn't; Step 3 shows the two are the same minimizer under a rescaled penalty constant.)

3. The ADMM stopping criterion (`norm_fn/dual_norm_fn` in `pipeline_nu.py`). This is the *only* place the genuine, physically-normalized Riemann-sum norm is computed – and it is computed slightly differently depending on which of the two variables it is handed:

$$\|x\|_{\text{diag}}^2 = \Delta x \sum_{j=1}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_j^{\text{dual}} \tilde{\rho}_{j,i+\frac{1}{2}}^2 + \Delta x \sum_{j=0}^{n_t-1} \sum_{i=1}^{n_x-1} \Delta t_{j+\frac{1}{2}} b_{j+\frac{1}{2},i}^2,$$

$$\Delta t_j^{\text{dual}} := \frac{1}{2} (\Delta t_{j-\frac{1}{2}} + \Delta t_{j+\frac{1}{2}}),$$

$$\|y\|_{\text{diag}}^2 = \Delta x \sum_{j=0}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_{j+\frac{1}{2}} \rho_{j+\frac{1}{2}, i+\frac{1}{2}}^2 + \Delta x \sum_{j=0}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_{j+\frac{1}{2}} m_{j+\frac{1}{2}, i+\frac{1}{2}}^2.$$

Δt_j^{dual} is the "dual cell" straddling interior edge j : half of neighboring cell $j - \frac{1}{2}$ plus half of neighboring cell $j + \frac{1}{2}$. But this norm is used *only* to decide when to stop iterating (comparing residuals D^k, P^k against tolerances) – it never appears in the formula that actually produces x^{k+1} or y^{k+1} .

1c. The one-line summary

where	grid	norm actually used
x -update (projection)	staggered	plain Euclidean, unweighted
y -update (KE prox, penalty term)	collocated	plain Euclidean, unweighted
y -update (KE prox, objective term)	collocated	relative $\Delta t_{j+\frac{1}{2}}/\Delta t_{\text{ref}}$ only
stopping criterion (<code>norm_fn</code>)	either	genuine $\Delta t \Delta x$ Riemann sum

The two proxies that actually define the iterates never use the physically-normalized norm. Only the diagnostic, sitting outside the iteration entirely, does.

Step 2: What norm approximates the continuous integral?

2a. The quadrature principle: primal cells vs. dual cells

The target is $\iint f(t, x)^2 dx dt$ over $[0, 1]^2$, and f is only known through samples at one of our three point sets (1a). Space is uniform throughout, so its half of the quadrature is always the trivial Δx per sample; the only real question is what weight a *time* sample gets, and that depends on *where in time* it sits.

A cell-centered sample. If f is represented as piecewise-constant on the primal partition $\{(t_j, t_{j+1})\}_{j=0}^{n_t-1}$ – i.e. sampled at the centers $t_{j+\frac{1}{2}}$, which is exactly how (ρ, m) live – the composite midpoint rule is

$$\int_0^1 f(t)^2 dt \approx \sum_{j=0}^{n_t-1} \Delta t_{j+\frac{1}{2}} f_{j+\frac{1}{2}}^2,$$

no derivation needed beyond “the sample *is* the cell average,” which is what a finite-volume field means.

An edge-valued sample. $\tilde{\rho}$ lives at interior time edges t_j , not centers – so what cell does $\tilde{\rho}_j$ represent an average *over*? The natural answer: the **dual cell** D_j centered at t_j and bounded by its two neighboring *primal cell centers*,

$$D_j := (\tau_{j-\frac{1}{2}}, \tau_{j+\frac{1}{2}}), \quad \tau_{j+\frac{1}{2}} := \frac{1}{2}(t_j + t_{j+1}).$$

These dual cells exactly tile $[0, 1]$ for the interior edges (each primal cell center is used as an endpoint by exactly two consecutive dual cells), so a midpoint rule on the *dual* partition is just as valid a quadrature as the one on the primal partition. Its width is

$$\begin{aligned} |D_j| &= \tau_{j+\frac{1}{2}} - \tau_{j-\frac{1}{2}} = \frac{1}{2}(t_j + t_{j+1}) - \frac{1}{2}(t_{j-1} + t_j) \\ &= \frac{1}{2}((t_{j+1} - t_j) + (t_j - t_{j-1})) = \frac{1}{2}(\Delta t_{j+\frac{1}{2}} + \Delta t_{j-\frac{1}{2}}) = \Delta t_j^{\text{dual}}, \end{aligned}$$

recovering §1’s Δt_j^{dual} *by derivation*, not by definition: it is exactly the distance between the two primal cell centers flanking edge t_j , which is the width of the one dual cell for which t_j is itself the midpoint.

$$\int_0^1 g(t)^2 dt \approx \sum_{j=1}^{n_t-1} \Delta t_j^{\text{dual}} g_j^2.$$

2b. The three target formulas

Applying this – primal weight $\Delta t_{j+\frac{1}{2}}$ when time-centered, dual weight Δt_j^{dual} when time-edge-valued, and Δx throughout in space, since space is always cell-centered here and never graded – gives the Riemann sum each variable *should* use to approximate its own continuous L^2 norm:

$$\begin{aligned} \|\rho\|^2, \|m\|^2 &\approx \Delta x \sum_{j=0}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_{j+\frac{1}{2}} \rho_{j+\frac{1}{2}, i+\frac{1}{2}}^2, & \Delta x \sum_{j=0}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_{j+\frac{1}{2}} m_{j+\frac{1}{2}, i+\frac{1}{2}}^2 & \text{ (collocated } y) \\ \|\tilde{\rho}\|^2 &\approx \Delta x \sum_{j=1}^{n_t-1} \sum_{i=0}^{n_x-1} \Delta t_j^{\text{dual}} \tilde{\rho}_{j, i+\frac{1}{2}}^2 & & \text{ (time-staggered)} \\ \|b\|^2 &\approx \Delta x \sum_{j=0}^{n_t-1} \sum_{i=1}^{n_x-1} \Delta t_{j+\frac{1}{2}} b_{j+\frac{1}{2}, i}^2 & & \text{ (space-staggered)} \end{aligned}$$

(b 's natural weight would be $\Delta t_{j+\frac{1}{2}} \Delta x_i^{\text{dual}}$ by the same primal/dual logic applied to space instead of time – but $\Delta x_i^{\text{dual}} = \frac{1}{2}(\Delta x + \Delta x) = \Delta x$ exactly, since space is uniform, so it silently collapses to the same Δx used everywhere else.)

2c. These are already §1's diagnostic norm

Comparing term by term: *this is exactly `norm_fn/dual_norm_fn`*, already derived in §1c rather than merely stated there. The stopping-criterion norm is not an ad hoc choice – it is the one genuine, derivable Riemann-sum approximation of $\iint f^2 dx dt$ for each of the three sample types.

where	grid	norm actually used	matches the target norm?
x -update (projection)	staggered	plain Euclidean, unweighted	no
y -update (KE prox, penalty)	collocated	plain Euclidean, unweighted	no
y -update (KE prox, objective)	collocated	relative $\Delta t_{j+\frac{1}{2}}/\Delta t_{\text{ref}}$	yes, exactly – see Step 3
stopping (norm_fn)	criterion either	genuine $\Delta t \Delta x$ Riemann sum	yes

Step 3: Substituting the target norm into the y -update (KE prox)

3a. A general fact about the scalar KE prox

Recall the pointwise kinetic-energy density, $J(\rho, m) = m^2/(2\rho)$ for $\rho > 0$ (the perspective function of $m^2/2$). Every version of the y -update below is, cell by cell, an instance of

$$L(\rho, m) = a J(\rho, m) + \frac{b}{2} [(\rho - \rho^z)^2 + (m - m^z)^2]$$

for some positive constants a (objective weight), b (penalty weight) – both possibly depending on j , but fixed once we are looking at one cell.

Lemma. The minimizer of L depends on a, b only through the ratio $\sigma := a/b$.

Proof. $\partial_m L = 0$: $\frac{a}{\rho}m + b(m - m^z) = 0 \Rightarrow m = \frac{b\rho m^z}{a + b\rho}$. $\partial_\rho L = 0$: $-\frac{a}{2\rho^2}m^2 + b(\rho - \rho^z) = 0$.
Substituting m and simplifying,

$$\rho - \rho^z = \frac{ab(m^z)^2}{2(a + b\rho)^2}.$$

Write $\sigma := a/b$, so $a + b\rho = b(\sigma + \rho)$; the b^2 in numerator and denominator cancel:

$$\rho - \rho^z = \frac{\sigma(m^z)^2}{2(\sigma + \rho)^2}.$$

b has dropped out entirely – the stationarity condition, and hence (ρ, m) , depends on a, b only via σ . $\square$

This is exactly why `prox_ke_cc` takes one parameter `sigma`, not two: an overall positive scalar multiplying an *unconstrained* objective never moves its minimizer, and b is precisely that overall scalar once $\sigma = a/b$ is held fixed.

3b. One elementary fact, used twice

Everything below is one consequence of the Lemma’s proof, used in two different ways – *multiplying an entire objective by a positive constant never moves its minimizer*: $\arg \min g = \arg \min c g$ for any $c > 0$. (This is literally why b dropped out of the Lemma: $L = b \cdot [\sigma J + \frac{1}{2}P]$, and $\arg \min L = \arg \min [\sigma J + \frac{1}{2}P]$ regardless of b .)

Use 1: a row-dependent common factor cancels (recovers §3’s opening finding).
Write $a_j = c_j \tilde{a}$, $b_j = c_j \tilde{b}$ for a row-dependent c_j but *row-independent* $\tilde{a}, \tilde{b}$. Then $\sigma_j = a_j/b_j = \tilde{a}/\tilde{b}$ – the *same* for every j , no matter how wildly c_j varies. This is exactly what happens when §2’s target norm, $\Delta t_{j+\frac{1}{2}} \Delta x$, weights *both* terms of the y -update:

$$(\rho, m)_{j+\frac{1}{2}, i+\frac{1}{2}}^{k+1} = \arg \min_{\rho, m} \Delta t_{j+\frac{1}{2}} \Delta x J(\rho, m) + \frac{\gamma}{2} \Delta t_{j+\frac{1}{2}} \Delta x P, \quad P := (\rho - \rho^z)^2 + (m - m^z)^2, \quad (1)$$

here $c_j = \Delta t_{j+\frac{1}{2}} \Delta x$, $(\tilde{a}, \tilde{b}) = (1, \gamma)$ – so $\sigma_j \equiv 1/\gamma$, and (1) collapses to the plain, unweighted, uniform-grid prox, no row-dependence at all.

Use 2: the code’s formula is J_h , not a compromise with it. Now apply the same elementary fact the *other* way: to one fixed row j , multiply the code’s whole objective by the single *global* constant $C := \Delta t_{\text{ref}} \Delta x$ (not row-dependent – just one number):

$$\begin{aligned} C \cdot \left[\frac{\Delta t_{j+\frac{1}{2}}}{\Delta t_{\text{ref}}} J(\rho, m) + \frac{\gamma}{2} P \right] &= \Delta t_{\text{ref}} \Delta x \cdot \frac{\Delta t_{j+\frac{1}{2}}}{\Delta t_{\text{ref}}} J(\rho, m) + \frac{\gamma \Delta t_{\text{ref}} \Delta x}{2} P \\ &= \underbrace{\Delta t_{j+\frac{1}{2}} \Delta x}_{\text{literal } J_h \text{'s weight}} J(\rho, m) + \frac{\hat{\gamma}}{2} P, \quad \hat{\gamma} := \gamma \Delta t_{\text{ref}} \Delta x. \end{aligned} \quad (2)$$

The Δt_{ref} ’s cancel exactly, leaving $\Delta t_{j+\frac{1}{2}} \Delta x$ – the literal, correct Riemann-sum weight for $\iint J dx dt$. Since $C > 0$ is a global constant, the left- and right-hand sides of (2) have the

same minimizer. So: **the code's formula, using penalty parameter γ , computes exactly what the literal, physically-correct J_h -weighted objective would compute if it used penalty parameter $\hat{\gamma} = \gamma \Delta t_{\text{ref}} \Delta x$ instead.** Not proportionally, not approximately – the identical minimizer, row by row.

3c. Punch line for the y -update

There are really only *two* distinct prox maps in this whole discussion, not three:

1. the **row-independent** map, $\sigma \equiv 1/\gamma$ – what (1) (weight everything by the target norm) collapses to;
2. the **genuinely row-dependent** map, $\sigma_j \propto \Delta t_{j+\frac{1}{2}}$ – which the code's formula and the literal J_h -weighted formula both compute, *exactly*, once their penalty constants are correctly identified ($\gamma \leftrightarrow \hat{\gamma} = \gamma \Delta t_{\text{ref}} \Delta x$).

The row-dependence the code has *is* the physically correct L^2 -quadrature weighting of the KE term – not a compromise that trades correctness for calibration. The only free choice is which of the two equivalent, rescaled penalty constants (γ or $\hat{\gamma}$) is the one you tune by hand; the code picks γ so that number stays $O(1)$ and resolution-independent, rather than requiring you to separately track the tiny, $n_t n_x$ -dependent $\hat{\gamma}$.

Step 4: Substituting the target norm into the x -update (projection)

4a. A general fact about weighted projection onto a linear constraint

The x -update solves $x^* = \arg \min_{x: Lx=d} \|x - v\|_W^2$ where $x = (\tilde{\rho}, b)$, W is the diagonal weight matrix encoding whatever norm we choose, and L is the (fixed, grid-dependent, but weight-independent) linear map built from D_t, A_t, D_x that encodes the Fokker–Planck constraint.

Lemma. $x^* = v - W^{-1}L^\top(LW^{-1}L^\top)^{-1}(Lv - d)$. If $W = cI$ for a single global scalar $c > 0$, this equals the *unweighted* ($W = I$) answer exactly.

Proof. Lagrangian $\frac{1}{2}(x - v)^\top W(x - v) + \lambda^\top(Lx - d)$; stationarity gives $W(x - v) + L^\top \lambda = 0$, so $x = v - W^{-1}L^\top \lambda$; substituting into $Lx = d$ gives $\lambda = (LW^{-1}L^\top)^{-1}(Lv - d)$, hence the formula. If $W = cI$, $W^{-1} = I/c$ and the two factors of $1/c$ (one in $W^{-1}L^\top$, one inside $(LW^{-1}L^\top)^{-1} = c(LL^\top)^{-1}$) cancel, leaving $v - L^\top(LL^\top)^{-1}(Lv - d)$ – independent of c . $\square$

This is the projection analogue of Step 3's Lemma – an overall global scalar is invisible – but notice the hypothesis is much stronger here: *every diagonal entry of W must be equal*, not just proportional row-by-row within an independent subproblem, because L mixes different coordinates of x together in a single equation. Step 2's target norm has W built from $\Delta t_{j+\frac{1}{2}}$ and Δt_j^{dual} , which are *not* constant across j once the grid is graded – so the hypothesis of the Lemma is not met, and there is no automatic reason to expect cancellation this time.

4b. Confirming it, concretely: the weighted adjoint of D_t

To make "no automatic reason" into a proof, compute what $L^\top$ becomes under W and check whether it is even proportional to the unweighted $L^\top$ (if it were, W would effectively act as a global rescaling on this piece, and Step 4a's cancellation would go through after all). Focus on D_t , the operator that actually changes under grading.

D_t maps $\tilde{\rho}$ (interior edges, index $j = 1, \dots, n_t - 1$) to a residual living at cell centers ($j + \frac{1}{2}$, $j = 0, \dots, n_t - 1$); its Euclidean transpose (ignoring the boundary terms, which carry no free variable) is, for interior j ,

$$(D_t^\top q)_j = \frac{q_{j-\frac{1}{2}}}{\Delta t_{j-\frac{1}{2}}} - \frac{q_{j+\frac{1}{2}}}{\Delta t_{j+\frac{1}{2}}}.$$

Lemma (kernel of $D_t^\top$). $\ker D_t^\top = \text{span}\{\Delta t\}$ (the vector with entries $\Delta t_{j+\frac{1}{2}}$), not $\text{span}\{1\}$, unless the grid is uniform.

Proof. $(D_t^\top q)_j = 0$ for every j means $q_{j-\frac{1}{2}}/\Delta t_{j-\frac{1}{2}} = q_{j+\frac{1}{2}}/\Delta t_{j+\frac{1}{2}}$ for every interior j , i.e. $q_{j+\frac{1}{2}}/\Delta t_{j+\frac{1}{2}}$ is the same constant C for every j , i.e. $q_{j+\frac{1}{2}} = C \Delta t_{j+\frac{1}{2}}$. $\square$

Now weight domain (edges) by Δt^{dual} and codomain (cells) by Δt – exactly Step 2's target norm, Δx dropped as always since it is a shared global constant. The weighted adjoint is $D_t^{*W}(q)_j := \frac{1}{\Delta t_j^{\text{dual}}}(D_t^\top[\Delta t \odot q])_j$; since $\Delta t_{j\pm\frac{1}{2}} \cdot q_{j\pm\frac{1}{2}}/\Delta t_{j\pm\frac{1}{2}} = q_{j\pm\frac{1}{2}}$, the Δt 's inside cancel and

$$D_t^{*W}(q)_j = \frac{q_{j-\frac{1}{2}} - q_{j+\frac{1}{2}}}{\Delta t_j^{\text{dual}}}.$$

Its kernel: $D_t^{*W}(q)_j = 0 \Leftrightarrow q_{j-\frac{1}{2}} = q_{j+\frac{1}{2}}$ for every $j \Leftrightarrow q \equiv \text{const}$, i.e. $\ker D_t^{*W} = \text{span}\{1\}$.

$D_t^\top$ and D_t^{*W} have different kernels whenever the grid is non-uniform ($\text{span}\{\Delta t\} \neq \text{span}\{1\}$ unless Δt is constant), and two linear maps with different kernels cannot be positive scalar multiples of one another. So D_t^{*W} is not "the same operator, just rescaled" – it is a genuinely different operator, and Step 4a's Lemma does not apply. This is the rigorous version of "the target norm does not cancel out of the x -update": unlike Step 3, there is no algebraic shortcut available here.

4c. Punch line for the x -update

Step 3 and Step 4 differ for a structural reason, not a coincidental one. The y -update is $n_t n_x$ unconstrained scalar problems; a weight that is constant *within each one* is always invisible, no matter how it varies *across* them. The x -update is *one* constrained problem; the constraint $Lx = d$ physically ties together coordinates that Step 2's target norm weights differently ($\tilde{\rho}$ by Δt^{dual} , b by Δt , both varying with j), so there is no shared scalar to divide out. Substituting the target norm here is not a free, cosmetic change the way it was for the KE prox – it produces an honestly different operator, D_t^{*W} , with a different kernel, and (as flagged in the companion note's Remark on the gauge mismatch) that operator does not preserve the projection's own well-posedness when substituted in.

Stopping here for Step 4. Step 5 takes up the remaining question: under this same target-norm convention, are $\mathcal{A}$ and $\mathcal{A}^\top$ (the consensus map linking x and y) still adjoint to each other?

Step 5: Is $\mathcal{A}$ still adjoint to $\mathcal{A}^\top$ under the target norm?

5a. The consensus map's time-averaging piece, today

`ladmm.py`'s x -update gradient step is `g = At_fn(gamma*r - delta)` – computed *every* LADMM iteration, not just inside the projection's precomputed system. Its time half is A_t , mapping $\tilde{\rho}$ (interior edges $j = 1, \dots, n_t - 1$, plus fixed boundary μ, ν) to the collocated grid by averaging the two edges bounding each cell: linear part $(A_t v)_{j+\frac{1}{2}} = \frac{1}{2}v_j + \frac{1}{2}v_{j+1}$ ($j = 0, \dots, n_t - 1$, boundary terms μ, ν dropped as before). Collecting the coefficient of v_j across the two cells it touches gives the Euclidean transpose

$$(A_t^\top q)_j = \frac{1}{2}q_{j-\frac{1}{2}} + \frac{1}{2}q_{j+\frac{1}{2}},$$

the plain, unweighted average of q 's two neighboring cells – exactly `interp_t_at_rho`, exactly what `At_fn` calls today.

5b. The weighted adjoint

Same recipe as Step 4b: weight the edge-space domain by Δt^{dual} , the cell-space codomain by Δt ,

$$A_t^{*W}(q)_j := \frac{1}{\Delta t_j^{\text{dual}}} (A_t^\top [\Delta t \odot q])_j = \frac{\frac{1}{2}\Delta t_{j-\frac{1}{2}}q_{j-\frac{1}{2}} + \frac{1}{2}\Delta t_{j+\frac{1}{2}}q_{j+\frac{1}{2}}}{\Delta t_j^{\text{dual}}} = \frac{\Delta t_{j-\frac{1}{2}}q_{j-\frac{1}{2}} + \Delta t_{j+\frac{1}{2}}q_{j+\frac{1}{2}}}{\Delta t_{j-\frac{1}{2}} + \Delta t_{j+\frac{1}{2}}}.$$

This is a genuine *weighted* average of q 's two neighbors – biased toward whichever neighboring cell is *wider* – not the plain $\frac{1}{2}, \frac{1}{2}$ average $A_t^\top$ computes.

5c. They are not the same operator

A_t^{*W} reduces to $A_t^\top$ exactly when $\Delta t_{j-\frac{1}{2}} = \Delta t_{j+\frac{1}{2}}$ (the two cells straddling edge j happen to share the same width). On a genuinely graded grid this fails at essentially every interior edge. More to the point: no single global constant c can make $A_t^{*W} = c A_t^\top$, because the two operators' *relative* weighting of $q_{j-\frac{1}{2}}$ vs $q_{j+\frac{1}{2}}$ differs – $A_t^\top$ always weighs them 1 : 1; A_t^{*W} weighs them $\Delta t_{j-\frac{1}{2}} : \Delta t_{j+\frac{1}{2}}$, a ratio that changes from edge to edge. A fixed 1 : 1 ratio scaled by any single c can never match a ratio that varies with j .

So no: $\mathcal{A}$ and $\mathcal{A}^\top$, as coded, are adjoint to each other only under the plain Euclidean inner product – not under Step 2's target norm.

5d. What this widens

The space half of $\mathcal{A}$ needs no such check: $\Delta x_i^{\text{dual}} = \frac{1}{2}(\Delta x + \Delta x) = \Delta x$ identically (space is never graded here), so its weight matrix is literally $\Delta x \cdot I$ – a global scalar – and Step 4a's Lemma applies directly: the space-averaging adjoint is unaffected.

Only the time-averaging piece changes, but it appears in *two* places, not one: inside the projection's precomputed M^k system (Step 4, already flagged as the gauge-mismatch Remark in the companion note), *and* directly inside `At_fn`, called fresh at every single LADMM iteration's gradient step. Making the scheme target-norm-consistent is therefore not a one-line fix confined to the projection – it touches the gradient computation that drives every x -update, too.

Stopping here for Step 5.

Step 6: Why doesn't the penalty need to be physically weighted too?

Step 3 showed the y -update's *objective* is exactly J_h – a genuine approximation of $\iint J dx dt$. Its *penalty* is still the plain, unweighted Euclidean P . That looks like an unresolved asymmetry: one term of the same optimization is measured in physical units, the other is not. It is not an oversight – here is why.

6a. The penalty term does not approximate anything continuous

The *true* problem this whole scheme discretizes is: minimize $\iint J dx dt$ over (ρ, m) satisfying the Fokker–Planck PDE, exactly. There is no " $Ax = y$ " in that problem at all – x (staggered) and y (collocated) are the *same* physical field; splitting it into two copies linked by a consensus constraint, and penalizing $\|Ax - y\|^2$, is purely a discrete *algorithmic* device introduced so ADMM can alternate between the projection and the KE prox. In the continuum this term is not merely small – it does not exist. So there is no continuous integral for the penalty to approximate, unlike J_h , which approximates a term that genuinely is in the original problem. Asking "what is the $\Delta t \Delta x$ -correct version of the penalty" is a category error: correctness for J_h means *consistency with the continuum*; correctness for the penalty means something else entirely – see 6b.

6b. What the penalty does need: one metric, shared by both updates

$(\gamma/2)\|Ax - y\|^2$ is *one* term, split by ADMM into a gradient contribution on the x side and a prox contribution on the y side – `ladmm.py`:

$$\underbrace{g = \text{At_fn}(\gamma r - \delta)}_{x\text{-side: gradient of the penalty}}, \quad \underbrace{y = \arg \min_y f_2(y) + \frac{\gamma}{2}\|y - z\|^2}_{y\text{-side: prox against the penalty}}.$$

g is exactly $\nabla_x \left[\frac{\gamma}{2} \|Ax + By - b\|^2 \right] + \mathcal{A}^\top \delta$ computed with $\mathcal{A}^\top = \text{At_fn}$ – and Step 5 established `At_fn` is the *Euclidean* adjoint. So the x -side already measures this penalty with the plain, unweighted metric – the *same* plain, unweighted metric the y -side penalty P uses. The two halves of one shared term agree with each other. That agreement, not a match to any continuous quantity, is the actual requirement – and it is exactly what today's code has.

6c. Weighting only the y -side would break that agreement

Multiplying P alone by $\Delta t_{j+\frac{1}{2}} \Delta x$ (§3, (1)) – without also changing `At_fn` on the x -side to match – would make the two halves of ADMM implicitly extremize *different*-metric versions of what is supposed to be one shared penalty. That is a strictly worse inconsistency than the current one, not an improvement. Making it consistent on *both* sides is exactly Step 4/5's target-norm-everywhere program, which needs $\mathcal{A}^{*W} \neq c \cdot \mathcal{A}^\top$ (Step 5c) and $D_t^{*W} \neq c \cdot D_t^\top$ (Step 4b) – already proved to break the projection's own algebra.

6d. Punch line

The asymmetry is the correct resolution, not a leftover compromise: the objective (J_h) has a continuum quantity to be faithful to, and is; the penalty has no continuum quantity to be faithful to, only an internal-consistency requirement between the x -update and y -update, and it meets that requirement (Euclidean, both sides). Making the penalty $\Delta t \Delta x$ -weighted on *both* sides to match J_h is not forbidden in principle – it is exactly the thing Steps 4–5 already showed is currently unavailable, because the operator that would have to change ($D_t^\top, A_t^\top$ inside the projection) cannot be replaced without losing the algebraic identity the projection relies on.

Stopping here for Step 6. This closes the loop back to Remark 3 (the gauge mismatch) in the companion note – everything in Steps 4–6 is really one obstruction, seen from three angles (the projection’s kernel, the consensus map’s adjoint, and now the shared penalty’s consistency).